

# Node JS安装

```
1  nvm install v26.3.0  #安装NodeJs
2  nvm use v26.3.0
3  nvm list          #查看当前环境
4  nvm use [env]    #切换当前环境
5
```

## Vite

### 官网

<http://cn.vitejs.dev>

## 创建vite项目

```
1  yarn create vite
```

## 安装

### Yarn安装

```
1  npm i -g yarn  #-g 全局  i 安装
2
3  yarn create vite #yarn 安装vite
4
5  yarn run dev #yarn启动
6  yarn run dev --host #暴露到外部调用
```

## Npm安装

```
1 npm create vite@latest
```

## Npm启动

```
1 npm run dev
```

## 目录与代码分析

### 工程目录分析

```
1 public:静态资源
2 src: 源代码
3 node_modules:工具库(第三方工具会安装到这里) 删除以后可以通过yarn 或 npm i 命令重新安装
4 index.html :项目入口文件，浏览器会优先加载该文件
5 package.json : 项目说明与依赖清单。
6 README.MD : 项目说明文件，说明项目如何启动，打包。
7 vite.config.js:项目配配置文件如端口号。
```

### 基础代码分析

```
1 import { createApp } from 'vue' #导入Vue包中函数
2 import './style.css'           #引入样式
3 import App from './App.vue'    #引入App模块
4 createApp(App).mount('#app')    #App模块挂载到#app HTML标签下。
```

### 模块基本结构

```
1 <template>
2   //html
3 </template>
4 <script setup lang="ts">
5
6   //脚本
7 </script>
8 <style scoped>
9   //css样式
10 </style>
```

## 代码编写

### 响应式与按钮操作

```
1 <template>
2   <div class="shop">
3     <h1>我的奶茶店</h1>
4     <p>剩余杯数:{{count}}杯</p>
5     <button @click="SellOne">卖出一杯了</button>
6     <div class="title">{{tishi}}</div>
7   </div>
8 </template>
9 <script setup lang="ts">
10  import { ref } from "vue"
11  var count = ref(10)
12  var tishi = ref('请选择要购买的商品')
13  function SellOne()
14  {
15    if(count.value>0)
16    {
17      count.value--
18    }
19    else
20    {
21      tishi.value="已经没有奶茶了"
22    }
23  }
24 </script>
25 <style scoped>
26  .shop {
27    text-align: center;
28    margin-top: 10px ;
29  }
30  h1{
31    color: #ff6b81;
32    font-size: 32px;
33  }
34  p {
35    color: #555;
36    font-size: 18px;
37  }
```

## 路由操作

## 下载依赖

```
1 yarn add vue-router@4
```

## 配置路由

src/router/index.ts

```
1 import {createRouter,createWebHistory} from "vue-router";
2 import Home from "../components/Home.vue"
3 import About from "../components/About.vue"
4
5 const routes = [
6   {
7     path: "/",
8     name: "首页",
9     component: Home
10  },
11  {
12    path: "/about",
13    name: "关于",
14    component: About
15  }
16 ]
17
18 const router = createRouter({
19   history:createWebHistory(),
20   routes
21 })
22
23 export default router;
```

# 使用路由

## App.vue

```
1
2 <template>
3   <router-view></router-view>
4   <!--
5     或者   <RouterView/>
6   -->
7 </template>
8
9 <script setup lang="ts">
10 </script>
11
12 <style scoped></style>
```

## src/mount.ts

```
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4 import router from './router/index'
5
6 const app = createApp(App)
7 app.use(router)
8 app.mount('#app')
9
```

# 页面跳转

## 申明式跳转

```
1 <router-link to="/about" target="_blank">关于</router-link>
2 <router-link to="/about">关于</router-link>
3 <RouterLink to="/">首页</RouterLink>
```

## 程式跳转

```
1 <script setup lang="ts">
2 import { useRouter } from "vue-router";
3 const router = useRouter();
4 function goToHome() {
5   router.push("/");
6 }
7 </script>
8 <template>
9   <h1>About Vue</h1>
10   <button @click="goToHome()">程式跳转</button>
11 </template>
12 <style scoped>
13 </style>
```

## 返回上一页

```
1 router.back(); 后退一页
2 router.go(-1); 后退一页
3 router.go(1);   前进一页
4 router.replace('/about') 页面替换 不能返回
```

## 数据绑定

### 静态绑定

```
1 <script setup lang="ts">
2   var UserName="张XX"
3 </script>
4 <template>
5   <div id="databind">
6     <p>我的名字是{{UserName}}</p>
7     <p>今天是{{UserName}}在打扫卫生</p>
8     <p>今天是{{UserName}}喝水</p>
9   </div>
10 </template>
11 <style scoped>
12 </style>
```

## 动态绑定

```
1 <script setup lang="ts">
2   import {ref} from 'vue';
3   var count=ref(0);
4
5   function Add()
6   {
7     count.value++;
8   }
9 </script>
10 <template>
11   <div id="databind">
12     {{count}}
13   </div>
14   <button style="width: 100px;height: 50px" @click="Add">加1</button>
15   <button style="width: 100px;height: 50px" @click="count--">减1</button>
16 </template>
17 <style scoped>
18 </style>
```

## 对象属性绑定

```
1 <script setup lang="ts">
2   import { reactive } from 'vue'
3   var User = reactive({ "name": "张三", "age": 18 })
4
5   function changeName()
6   {
7       User.name = "王五"
8   }
9   function addAge()
10  {
11      User.age++ ;
12  }
13
14 </script>
15 <template>
16   <div id="databind">
17       {{User.name}} {{User.age}} 岁
18   </div>
19   <button style="width: 100px;height: 50px" @click="changeName">改名字</button>
20   <button style="width: 100px;height: 50px" @click="addAge">年龄+1</button>
21 </template>
22 <style scoped>
23 </style>
```

## 双向绑定

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3   var msg = ref("")
4 </script>
5 <template>
6   <div id="binddouble">
7     <input type="text" v-model="msg" placeholder="请输入内容">
8     <p>你输入的内容是{{msg}}</p>
9   </div>
10 </template>
11 <style scoped>
12 </style>
```

## 计算属性

```
1 <script setup lang="ts">
2   import { ref ,computed} from 'vue'
3   var firstName = ref("")
4   var lastName = ref("")
5   var fullName = computed(function (){
6     console.log("我在计算全名...")
7     return firstName.value+" "+lastName.value
8   })
9 </script>
10 <template>
11   <div id="binddouble">
12     FirstName:<input type="text" v-model="firstName" placeholder="请输入内容">
13     Lastname:<input type="text" v-model="lastName" placeholder="请输入内容">
14     <p>你的全名是{{fullName}}</p>
15     <p>你的全名是{{fullName}}</p>
16     <p>你的全名是{{fullName}}</p>
17     <!-- 只计算一次 -->
18   </div>
19 </template>
20 <style scoped>
21 </style>
```

## 监听器绑定

```
1 <script setup lang="ts">
2   import { ref ,watch} from 'vue'
3   var age = ref("18")
4   function groupth()
5   {
6     age.value++
7   }
8   watch(age,(newVal,oldVal)=>{
9     console.log(`年龄变量，新只是${newVal} 老值是${oldVal}`)
10  })
11 </script>
12 <template>
13   <div id="binddouble">
14     年龄:{{age}} <button @click="groupth()">长大一岁</button>
15   </div>
16 </template>
17 <style scoped>
18 </style>
```

## 插值表达式

```
1 <script setup lang="ts">
2   var currAge = 18
3   var firstname = "zhang"
4   var lastname = "san"
5 </script>
6 <template>
7   <div id="binddouble">
8     年龄:{{currAge}}
9     去年年龄:{{currAge-1}}
10    明年年龄:{{currAge+1}}
11    <br>
12    {{firstname}} -----{{lastname}}-----{{firstname+"-"+lastname}}
13  </div>
14 </template>
15 <style scoped>
16 </style>
```

## 图片绑定

```
1 <script setup lang="ts">
2 import {ref} from 'vue'
3 var imgpath = ref('/img01/1.webp') ;
4 var imageArr = ['/img01/1.webp', '/img01/2.webp', '/img01/3.webp'] ;
5 var index = 0 ;
6 setInterval(function () {
7   imgpath.value= imageArr[index++%imageArr.length]
8 },1000)
9 </script>
10 <template>
11   <div id="binddouble">
12     
13     
14   </div>
15 </template>
16 <style scoped>
17 </style>
```

## 类名绑定

```
1 var clsName = "testClsName"
2
3 <div :class="clsName">修改Class类名</div>
4 <div v-bind:class="clsName">修改Class类名</div>
```

## 条件渲染

```
1 <script setup lang="ts">
2   import { ref } from 'vue'
3   var isLogin = true
4   var isShow = ref(false)
5 </script>
6 <template>
7   <div id="binddouble">
8     <p v-if="isLogin">欢迎你，已经登录成功</p>
9     <p v-else>请先登录账号</p>
10    <p v-show="isShow">我是可以隐藏的文字</p>
11    <button @click="isShow=!isShow">切换显示与隐藏</button>
12  </div>
13 </template>
14 <style scoped>
15 </style>
```

## 列表渲染

### 简单列表

```
1 <script setup lang="ts">
2   var list = ['AA','BB','CC','DD','EE']
3 </script>
4 <template>
5   <div id="binddouble">
6     <div id="simpleList">
7       <ul>
8         <li v-for="item in list" key="item">{{item}}</li>
9       </ul>
10    </div>
11  </div>
12 </template>
13 <style scoped>
14 </style>
```

对象列表

```
1 <script setup lang="ts">
2   var students = [
3     {
4       id:1,
5       name:'zs',
6       age: 18,
7       sex: '男'
8     },
9     {
10      id:2,
11      name:'lishi',
12      age: 20,
13      sex: '男'
14    },
15    {
16      id:3,
17      name:'hanmeimei',
18      age: 22,
19      sex: '女'
20    }
21  ]
22 </script>
23 <template>
24   <div id="binddouble">
25     <div id="simpleList">
26       <table>
27         <tr v-for="item in students" key="item.id">
28           <td>{{item.id}}</td>
29           <td>{{item.name}}</td>
30           <td>{{item.age}}</td>
31           <td>{{item.sex}}</td>
32         </tr>
33       </table>
34     </div>
35   </div>
36 </template>
37 <style scoped>
```

## 事件修饰符

### <a>标签禁止跳转

```
1 <script setup lang="ts">
2   function sayHi()
3   {
4     alert("Hello !")
5   }
6 </script>
7 <template>
8   <div id="event001">
9     <p><a href="https://www.baidu.com">点我跳转</a></p>
10    <p><a href="https://www.baidu.com" @click.prevent="sayHi()">点我不跳转</a></p>
11  </div>
12 </template>
13 <style scoped>
14 </style>
```

### 阻止事件冒泡

```
1 <script setup lang="ts">
2   function fatherClick()
3   {
4     console.log('父盒子被点击了')
5   }
6   function sonClick()
7   {
8     console.log('子盒子被点击了')
9   }
10 </script>
11 <template>
12   <div id="event002">
13     <div class="father" @click="fatherClick()">
14       我是父盒子
15       <button @click="sonClick()">我是子按钮1</button>
16       <br>
17       <button @click.stop="sonClick()">我是子按钮2</button>
18     </div>
19   </div>
20 </template>
21 <style scoped>
22 </style>
23
```

## 一次性事件

```
1 <script setup lang="ts">
2   function clickOnce()
3   {
4     alert("我被点击了")
5   }
6 </script>
7 <template>
8   <div id="event002">
9     <button @click.once="clickOnce()">只能点击一次的按钮</button>
10  </div>
11 </template>
12 <style scoped>
13 </style>
```

## 案件修饰符

```
1 <script setup lang="ts">
2   let msg=""
3   function submit()
4   {
5     alert("执行提交... : "+msg)
6   }
7 </script>
8 <template>
9   <div id="event002">
10     <input type="text"
11       placeholder="搜索你想找的内容"
12       v-model="msg"
13       @keyup.enter="submit()">
14   </div>
15 </template>
16 <style scoped>
17 </style>
```

## 修饰符链式调用

```
1 <a href="" @click.prevent.stop="fn()">既不跳转也不冒泡</a>
```

# 组件

## 引入组件

### 内容栏

```
1 <script setup lang="ts">
2
3 </script>
4
5 <template>
6   <div class="nav">我是内容栏</div>
7 </template>
8
9 <style scoped>
10  .nav
11  {
12    width: 100%;
13    height: 800px;
14    background-color: #c8b1e4;
15  }
16 </style>
```

### 标题栏

```
1 <script setup lang="ts">
2
3 </script>
4
5 <template>
6   <div class="nav">我是标题栏</div>
7 </template>
8
9 <style scoped>
10  .nav
11  {
12    width: 100%;
13    height: 80px;
14    background-color: #7f46c7;
15  }
16 </style>
```

## 导航栏

```
1 <script setup lang="ts">
2
3 </script>
4
5 <template>
6   <div class="nav">我是导航栏</div>
7 </template>
8
9 <style scoped>
10  .nav
11  {
12    width: 100%;
13    height: 40px;
14    background-color: #6b6375;
15  }
16 </style>
```

## 页面引入

```
1 <script setup lang="ts">
2 import Header from "../dir01/Header.vue"
3 import Nav from "../dir01/Nav.vue"
4 import Content from "../dir01/Content.vue"
5 </script>
6 <template>
7   <Header></Header>
8   <Nav></Nav>
9   <Content></Content>
10 </template>
11 <style scoped>
12 </style>
```

## 全局引入

```
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4 import router from "../router/index"
5
6
7
8 import Nav from './components/dir01/Nav.vue'
9 import Header from './components/dir01/Header.vue'
10 import Content from './components/dir01/Content.vue'
11
12
13 const app = createApp(App)
14 app.use(router)
15 app.mount('#app')
16 app.component('my-nav',Nav)
17 app.component('Header',Header)
18 app.component('Content',Content)
```

直接使用，不用单独再引入

```
1 <script setup lang="ts">
2 </script>
3 <template>
4   <Header></Header>
5   <my-nav></my-nav>
6   <Content></Content>
7 </template>
8 <style scoped>
9 </style>
```

## 父到子组件传值

子组件Sub.vue

```
1 <script setup lang="ts">
2 const props = defineProps({
3   message:{
4     type:String,
5     required:true,
6     default:"默认标题"
7   }
8 })
9 </script>
10 <template>
11   <div class="container">
12     <h1>{{message}}</h1>
13   </div>
14 </template>
15
16 <style scoped>
17   .container
18   {
19     width: 100%;
20     height: 100px;
21     background-color: #9ca3af;
22   }
23   .container h1
24   {
25     text-align: center;
26     font-size: 30px;
27     color: red;
28   }
29 </style>
```

## 父组件

```
1 <script setup lang="ts">
2 import Sub from './Sub.vue'
3 </script>
4
5 <template>
6   <Sub message="测试标题"></Sub>
7 </template>
8
9 <style scoped>
10 </style>
```

## 子到父组件传值

子组件Sub.vue

```
1 <script setup lang="ts">
2 var emit = defineEmits(["btnClick"])
3 function btnClick()
4 {
5   emit("btnClick","我是传给父组件的数据");
6 }
7 </script>
8
9 <template>
10   <div id="container">
11     子组件
12     <button @click="btnClick()">点击我给父组件发消息</button>
13   </div>
14 </template>
15
16 <style scoped>
17
18 </style>
```

父组件

```
1 <script setup lang="ts">
2 import {ref} from 'vue'
3 import Child from './Sub.vue'
4 var fathermsgs = ref("")
5 function getSonData(msg)
6 {
7   fathermsgs.value=msg
8 }
9 </script>
10
11 <template>
12   <div id="container">
13     父组件:{{fathermsgs}}
14     <Child @btnClick="getSonData"></Child>
15   </div>
16 </template>
17 <style scoped>
18 </style>
```

## 插槽

### 匿名插槽

子组件

```
1 <script setup lang="ts">
2
3 </script>
4
5 <template>
6   <div id="sub_container">
7     <slot>我是插槽默认显示内容</slot>
8   </div>
9 </template>
10
11 <style scoped>
12
13 </style>
```

## 父组件

```
1 <script setup lang="ts">
2 import Sub from "./Sub.vue";
3 </script>
4 <template>
5   <div id="container_parent">
6     <Sub>
7       <!--有内容就替换插槽内容，没有就展示插槽默认内容-->
8       <button>我是按钮</button>
9       <p>
10         我是换行
11       </p>
12     </Sub>
13   </div>
14 </template>
15 <style scoped>
16 </style>
```

## 命名插槽(指定替换)

### 父组件

```
1 <script setup lang="ts">
2 import Sub from "./Sub.vue";
3 </script>
4 <template>
5   <div id="container_parent">
6     <Sub>
7       <template #header>
8         <h1>abcd</h1>
9       </template>
10    </Sub>
11  </div>
12 </template>
13 <style scoped>
14 </style>
```

## 子组件

```
1 <script setup lang="ts">
2
3 </script>
4
5 <template>
6   <div id="sub_container">
7     <div class="content">
8       <slot><h1>我是插槽默认显示内容xx</h1></slot>
9     </div>
10    <div class="content">
11      <slot name="header">
12        <h1>我是标题</h1>
13      </slot>
14    </div>
15    <div class="content">
16      <slot name="footer">
17        <h1>我是底部内容</h1>
18      </slot>
19    </div>
20    <div class="content">
21      <slot name="body">
22        <h1>主题内容</h1>
23      </slot>
24    </div>
25  </div>
26 </template>
27
28 <style scoped>
29   h1
30   {
31     width: 100%;
32     height: 200px;
33     color: #b1381e;
34   }
35 </style>
```

## 生命周期

## 创建和挂载阶段

```
1 <script setup lang="ts">
2 import { onMounted } from 'vue';
3
4 console.log("组件创建")
5
6 onMounted(function (){
7   console.log("onMounted已经执行，可以获取页面元素了")
8   var dom = document.querySelector(".title")
9   console.log(dom)
10 })
11 </script>
12
13 <template>
14   <div id="container">
15     <h1 class="title"></h1>
16   </div>
17 </template>
18
19 <style scoped>
20
21 </style>
```

## 更新阶段

```
1 <script setup lang="ts">
2 import { onBeforeUpdate, ref } from 'vue';
3 var count = ref("0")
4 onBeforeUpdate(function (){
5   console.log("数据已经更新了")
6 })
7 </script>
8 <template>
9   <div id="container">
10     <div>{{count}}</div>
11     <button @click="count++">AAA</button>
12   </div>
13 </template>
14 <style scoped>
15 </style>
```

## 销毁阶段

关闭监听事件，取消网络请求

```
1 <script setup lang="ts">
2 import { onBeforeUpdate, ref, onUnmounted } from 'vue';
3 onUnmounted(function (){
4   console.log("组件销毁了.....")
5 })
6 </script>
```